



CI/CD Using AWS

Introduction

CI/CD (**Continuous Integration and Continuous Deployment**) on **AWS**

automates the software development lifecycle by integrating code changes, testing, and deploying applications seamlessly.

- **What is CI/CD in AWS?**
- **Why use AWS for CI/CD?**
- **How to implement CI/CD pipelines on AWS?**

This guide explains **how AWS enables automated CI/CD pipelines** using its cloud-based tools.

Section 1: Learn

What is CI/CD on AWS?

CI/CD on AWS **automates the software build, testing, and deployment processes** using services like:

- **AWS CodeCommit** (Git repository for storing code)
- **AWS CodeBuild** (Compiles and tests application code)
- **AWS CodeDeploy** (Deploys applications to EC2, Lambda, or on-prem servers)
- **AWS CodePipeline** (Manages the entire CI/CD workflow)



Why Use AWS for CI/CD?

Benefit	Description
Scalability	Runs CI/CD pipelines on AWS cloud without infrastructure setup
Speed	Reduces deployment time with automation
Security	AWS IAM provides fine-grained access control
Integration	Works with GitHub, Bitbucket, and third-party tools
Cost-Efficiency	Pay only for what you use

CI/CD Workflow on AWS

Step	Service Used	Purpose
Step 1: Commit Code	AWS CodeCommit	Developers push code changes
Step 2: Build & Test	AWS CodeBuild	Code is compiled and tested automatically
Step 3: Deploy to Staging	AWS CodeDeploy	Deploys to a test environment
Step 4: Approval Process	AWS CodePipeline	Manual approval before production
Step 5: Deploy to Production	AWS CodeDeploy	Releases application to live users



Interesting Fact

Companies like **Netflix, Airbnb, and LinkedIn** use **AWS CI/CD tools** to deploy updates **several times per day**.

Section 2: Practice

1. Setting Up a CI/CD Pipeline on AWS

Step 1: Create a Git Repository in AWS CodeCommit

1. Open **AWS Management Console** → Navigate to **CodeCommit**.
2. Click **Create Repository** → Enter **Repository Name** → Click **Create**.
3. Push your code using:

```
git clone
https://git-codecommit.<AWS_REGION>.amazonaws.com/v1/repos/MyRepo
cd MyRepo
git add .
git commit -m "Initial commit"
git push origin master
```

Step 2: Create a Build Project in AWS CodeBuild

1. Navigate to **AWS CodeBuild** → Click **Create Build Project**.
2. Enter **Project Name** → Select **Source Provider: AWS CodeCommit**.
3. Choose a **Build Environment** (e.g., Ubuntu, Amazon Linux).
4. Define a **buildspec.yml** file in your repository:

```
version: 0.2
```

```
phases:
```



install:

runtime-versions:

java: corretto11

build:

commands:

- mvn clean install
- mvn test

2. Click **Create Build Project** → Start the Build.

Step 3: Deploy Using AWS CodeDeploy

1. Navigate to **AWS CodeDeploy** → Click **Create Application**.
2. Select **Compute Platform** (EC2, Lambda, or ECS).
3. Define a **appspec.yml** file for deployment:

version: 0.0

os: linux

files:

- source: /
destination: /var/www/html/

hooks:

BeforeInstall:

- location: scripts/before_install.sh
timeout: 300

AfterInstall:

- location: scripts/after_install.sh
timeout: 300



2. Click **Create Deployment** → Select **EC2 Instances** → Click **Deploy Now**.

Step 4: Automate the Pipeline with AWS CodePipeline

1. Navigate to **AWS CodePipeline** → Click **Create Pipeline**.
2. Choose **Source Stage** → Select **AWS CodeCommit**.
3. Choose **Build Stage** → Select **AWS CodeBuild**.
4. Choose **Deploy Stage** → Select **AWS CodeDeploy**.
5. Click **Create Pipeline**.

Now, every code push will automatically trigger the CI/CD pipeline.

Why is this useful?

- Reduces **manual work** and speeds up software deployment.
-

2. Deploying a Serverless App Using AWS Lambda

CI/CD can also **deploy serverless applications**.

Step 1: Configure AWS CodePipeline for Lambda

1. Open **AWS CodePipeline** → Click **Create Pipeline**.
2. Choose **Source** → Select **GitHub/AWS CodeCommit**.
3. Choose **Build Stage** → Select **AWS CodeBuild**.
4. Choose **Deploy Stage** → Select **AWS Lambda**.

Step 2: Deploy AWS Lambda Using CodeDeploy

Modify **appspec.yml** for Lambda:

```
version: 0.0

Resources:

- TargetService:

  Type: AWS::Lambda::Function
```



Properties:

Name: MyLambdaFunction

Alias: live

CurrentVersion: 1

Why is this important?

- Enables **zero-downtime deployments** for **serverless applications**.
-

Section 3: Know More

Frequently Asked Questions

1. **What is the difference between CI and CD?**
 - **CI** automates code integration and testing.
 - **CD** automates application deployment to production.
 2. **Can AWS CI/CD work with GitHub?**
 - Yes! AWS CodePipeline can pull code from **GitHub, Bitbucket, and GitLab**.
 3. **How do I monitor my CI/CD pipeline in AWS?**
 - Use **AWS CloudWatch Logs** and **AWS CodePipeline Dashboard**.
 4. **What happens if a build fails in AWS CodeBuild?**
 - AWS logs errors, and the pipeline **stops execution automatically**.
 5. **Can I deploy Docker containers using AWS CI/CD?**
 - Yes! AWS CodePipeline can **deploy Docker containers to Amazon ECS and Kubernetes**.
-



Conclusion

CI/CD using AWS **automates code integration, testing, and deployment**, ensuring **faster and reliable software releases**.

Start by **creating AWS CodeCommit repositories, configuring CodeBuild, setting up CodeDeploy, and automating the process using CodePipeline**, and soon you'll be **deploying applications seamlessly on AWS!**